

Module #4

Tableaux et Claviers.

1. Lecture des interrupteurs: Détection Front et Anti-Rebond.....	1
1.1. La lecture des interrupteurs.	1
1.2. Technique logicielle, déjà vue, pour détecter Front avec Anti-rebond:..	2
1.3. Techniques matérielles:.....	2
1.4. Technique de l'anti-rebond par compteur:	4
2. Les Tableaux.....	5
2.1. Généralités:.....	5
2.2. Initialisation des tableaux	6
2.3. Tableau à plusieurs dimensions	7
3. Les chaînes de caractères	8
4. Les Claviers.	9
4.1. Lecture du clavier de l'ordinateur (PC).....	9
4.2. Clavier à brancher	10
4.3. Clavier à matrice:.....	11
4.3.1. Décodé avec des circuits de bases	11
4.3.2. Décodé avec un port du microcontrôleur.....	12
4.3.3. Décodé avec un 74C922.....	13

1. Lecture des interrupteurs: Détection Front et Anti-Rebond.

1.1 La lecture des interrupteurs.

Lors de la lecture d'interrupteurs, doit-on toujours utiliser un anti-rebond et une détection de front ?

La réponse est évidemment non !

L'utilisation ou non, d'une ou des deux techniques dépend de l'application.

Exemple:

SI(le bouton est pesé)	Allume la Led.
SINON	Eteint la Led.

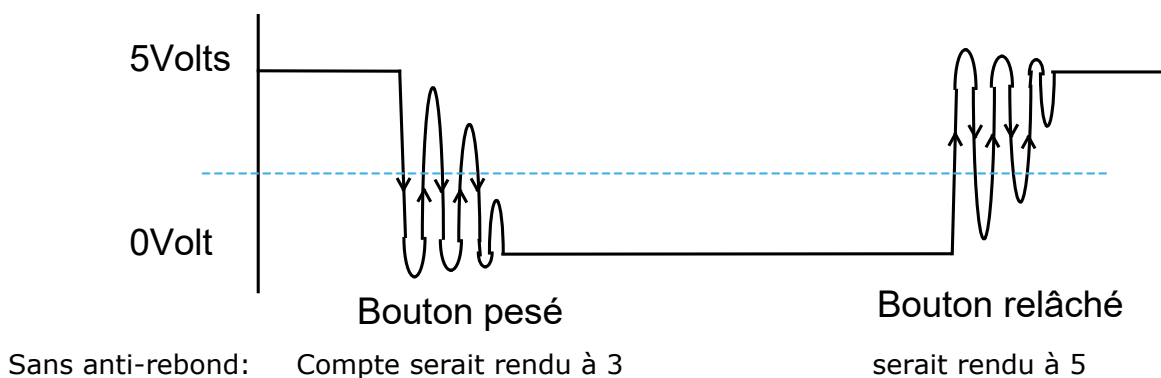
Ici, pas besoin d'aucune technique

Par contre dans cet exemple:

SI(le bouton est pesé)	Compte = Compte + 1.
------------------------	----------------------

Ici, il faudra détecter un front et même utiliser l'anti-rebond sous peine de voir augmenter le Compte beaucoup trop vite.

Exemple avec augmentation du compte quand on détecte seulement un front de descente:



1.2 Technique logicielle, déjà vue, pour détecter Front avec Anti-rebond:

```

SI(ucAncienEtat != ucBoutonMarche)    // Changement du bouton Marche?
    ucAncienEtat = ucBoutonMarche.    // Sauvegarde le nouvel etat.
    Delai de 10ms.                    // Anti-rebond (99% du programme).
    SI(ucBoutonMarche == 1)            // Front de monte?
        // Mettre ici le code a executer si le bouton a ete relache.
    SINON                                // Front de descente.
        // Mettre ici le code a executer si le bouton a ete pese.
    FIN SI.
FIN SI.
    
```

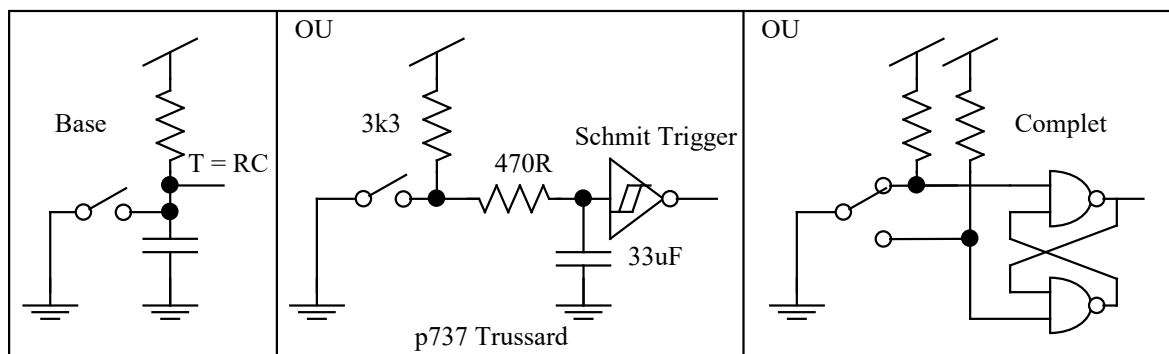
Le gros désavantage de cette technique est que nous restons pris dans le délai de 10ms. Et ce délai représente environ 99% du temps d'exécution de notre programme. En d'autres mots, on passe 99% de notre temps à ne rien faire.

Évidemment si on veut faire du contrôle à grande vitesse, cette technique n'est plus valable

1.3 Techniques matérielles:

Parfois, nous pouvons régler le problème de l'anti-rebond de façon matériel.

Voici quelques circuits:



Pour le choix de R et C: ($E = 5\text{Volts}$)

Charge (0 à 1):

$$V = E(1 - e^{-\text{temps}/RC})$$

$$V_{(RC)} = 0.63E \quad (\text{ici: } 0.63 \cdot 5 = 3.2\text{V})$$

$$\text{Temps} = -RC \cdot \ln(1 - V/E)$$

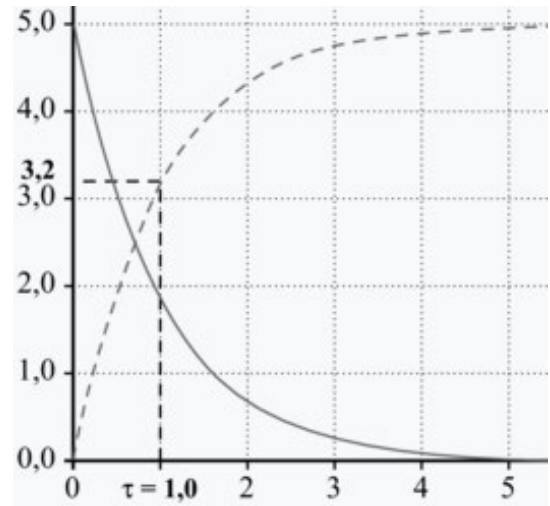
Décharge (1 à 0):

$$\text{Temps}_{1/2E} = RC \cdot \ln 2 \quad (1/2E = 2.5\text{V})$$

$$V_{(RC)} = 0.37E \quad (\text{ici: } 0.37 \cdot 5 = 1.85\text{V})$$

Donc on cherche $RC \approx 0.01$ (10ms).

Schmit: 1.6V: L'entrée passe au niveau Haut
0.8V: L'entrée passe au niveau Bas



Technique: Lorsqu'il y a changement de l'interrupteur, il faut trouver une valeur de RC qui nous permet de rester au même niveau logique durant au moins 10ms

Passage de 0 à 1. Il faut que le voltage à l'entré reste inférieur à 0.8V durant au moins 10ms.

Avec l'équation: $\text{Temps} = -RC \cdot \ln(1 - V/E)$

On trouve $RC = -\text{Temps} / \ln(1 - V/E) = -0.01\text{Sec} / \ln(1 - 0.8/5) = -0.01 / \ln(1 - 0.16)$

$$= -0.01 / \ln(0.84) = -0.01 / -0.1743 = 0.057 = RC$$

Si $C = 10\mu\text{f} \rightarrow R = 0.057 / 0.00001 = 5735 \text{ ohms}$. (5600 valeur standard 10%)

Donc, $R = 5.6\text{K}$ et $C = 10\mu\text{f}$.

Passage de 1 à 0. Il faut que le voltage à l'entré reste supérieur à 2V durant au moins 10ms.

Avec l'équation: $\text{Temps}_{1/2E} = RC \cdot \ln 2$ ($\text{Temps}_{1/2E} = 0.01\text{sec}$ pour avoir 2.5V)

On trouve $RC = 0.01\text{sec} / \ln 2 = 0.01\text{sec} / 0.693 = 0.0144 = RC$

Si $C = 10\mu\text{f} \rightarrow R = 0.0144 / 0.00001 = 1442 \text{ ohms}$

Donc, $R = 2\text{K}$ et $C = 10\mu\text{f}$ feraient l'affaire.

Pour avoir les mêmes valeurs dans les deux cas, on prend $C = 10\mu\text{f}$ et $R = 5.6\text{K}$.
Avec $R = 5.6\text{K}$ le passage du niveau 1 au niveau 0 sera tout simplement plus long.

Pour le Schmit (avec les valeurs du schéma):

(0 à 1) On a 0.38V après 10ms (formule: $V = E(1 - e^{-\text{temps}/RC})$, où: $RC = 3770 \cdot 33 \cdot 10^{-6} = 0.12441$)

(1 à 0) Après 10.7ms on est à 2.5V (formule: $\text{Temps}_{1/2E} = RC \cdot \ln 2$, où: $RC = 470 \cdot 33 \cdot 10^{-6} = 0.01551$)

1.4 Technique de l'anti-rebond par compteur:

Malheureusement, nous n'avons souvent pas accès aux circuits électroniques. Il nous faudrait donc une solution logicielle. Une manière simple d'attendre le 10ms sans bloquer dans un délai, est de compter le nombre de fois qu'on passe dans la boucle principale du programme lorsque le bouton a été pesé. Si ça fait 100 ou 200 fois qu'on passe dans la boucle, on peut penser que le rebondissement est terminé.

Voici un algorithme qui décrit le fonctionnement de l'anti-rebond par compteur:

```

Rebond = 0.
...
TANT QUE(VRAI)
...
  LIRE Boutons sur Port.
  SI(Boutons pesés)          //Un ou plusieurs boutons.
    SI(Rebond < 250)
      Rebond = Rebond + 1.
      SI(Rebond == 200)      // 200 Boucles? On espère ≈10ms.

```

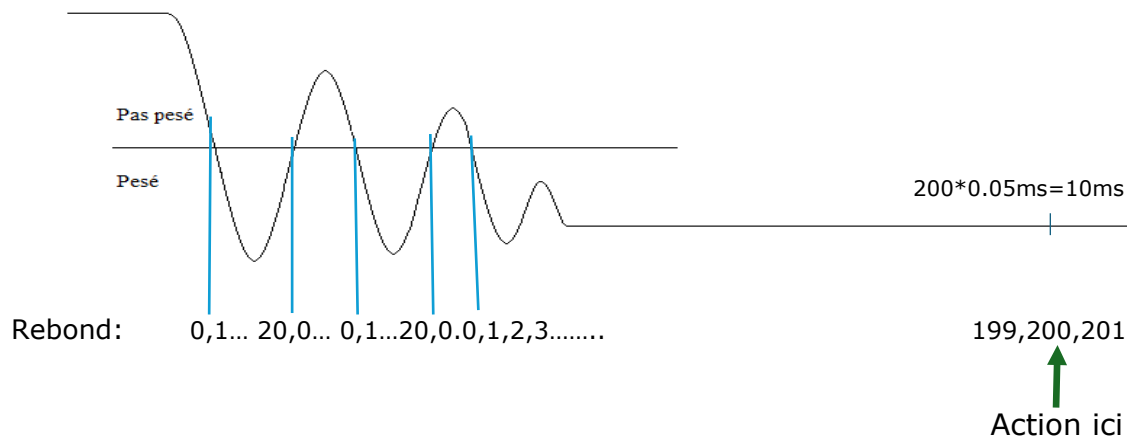
ICI, Action à faire quand rebondissement terminé.

```

  FIN SI.
  FIN SI.
  SINON                      // Bouton pas pese.
    Rebond = 0.              // On désactive anti-rebond.
  FIN SI.
...
FIN TANT QUE.

```

Exemple: La boucle principale de mon programme prend 50us (0.05ms) à s'exécuter.
Si je pèse sur un bouton voici ce à quoi ça ressemble une fois agrandi.
La largeur d'une pulse dure 1ms (1ms / 0.05ms = 20 boucles).



L'avantage de cette technique est que nous ne restons pas pris dans une boucle de délai.

L'inconvénient de la technique c'est le délai qui est difficile à calculer. Ça dépend de la boucle totale du programme. De plus, le temps d'exécution de la boucle du programme peut varier (changement aux entrées, affichage de message, etc...).

Pour avoir un délai précis on pourrait utiliser un Timer. Mais les Timers sont très utilisés en contrôle, il est préférable de les garder pour des applications spécifiques.

Le désavantage des anti-rebonds, c'est que s'ils sont mal ajustés, on peut perdre des informations pertinentes (un signal qui revient vite pourrait ne pas être lu). Ou avoir des rebonds si trop court.

2. Les Tableaux.

2.1 Généralités:

Un tableau est un regroupement, dans une **même variable**, de **plusieurs données**, de **même type**.

La déclaration: **Type Nom[NOMBRE_ELEMENTS];**

Exemple: unsigned char ucTab[10];

Ceci réserve, en mémoire, un espace contigu pouvant contenir 10 éléments (ici 10 octets).

Le premier élément est ucTab[0] et le dernier est ucTab[9] (de 0 à 9 donne 10 éléments).

ATTENTION: Il n'y a aucune vérification du débordement de tableau.

En utilisant ucTab[10] ou plus, aucune erreur ne sera signalée et vous utiliserez une partie de mémoire qui a certainement été réservée pour autre chose.

N'oubliez pas, avec les tableaux:

- **L'indice du premier élément est 0**
- **Le dernier élément du tableau est [NOMBRE_ELEMENTS - 1]**

L'identificateur ucTab est une variable spéciale, dont la valeur est l'adresse de ucTab[0].

Il ne sera donc pas possible de faire figurer ucTab dans le premier membre d'une affectation (Lvalue).

Exemple: `ucTab = 10; // Donne l'erreur "C213: left side of asn-op not an lvalue".`

`// asn-op:    assignment operator ( = ).`
 `// lvalue:    quelque chose qui peut être à gauche du signe =`
 `//                donc qui peut être modifié.`

2.2 Initialisation des tableaux.

Nous pouvons initialiser les tableaux au moment de leur déclaration. Cette façon de faire nous assure que la variable ne sera jamais utilisée sans être initialisée.

Ex.:

```
unsigned char ucTable[3]      = {'1', 0x32, 51};
                                // 3 éléments, 3 octets.

unsigned int  uiTableau[5]    = {1000, 2000, 3000, 4000, 5000};
                                // 5 éléments, 10 octets.

char  cTableauCaractere[10]  = {"Bonjour !"};
                                // 9 caractères, 10 éléments, 10 octets.
```

Note: En dehors de la déclaration, il n'est pas possible d'affecter une chaîne de caractère à un tableau. Il faudra faire une copie de la chaîne que nous voulons écrire dans le tableau octet par octet.

Si on initialise un tableau, alors, il n'est pas nécessaire de spécifier la dimension.

```
UI uiChiffres[]={0,1,2,3,4,5,6,7,8,9}; //Réserve 10 espaces mémoire
                                // (20 octets)
```

Mais si la dimension est donnée, et qu'elle est supérieure au nombre de valeurs données, les suivantes seront indéterminées.

```
UI uiTableau[20]={1,2,3}; // 3 espaces définis les 17 autres indéterminés.
```

Si la dimension du tableau est inférieure aux nombres de données, il va y avoir une erreur.

```
int iBad[2] = {1,2,3,4}; // error C242: 'array[]': too many initializers.
```

Si l'initialisation des valeurs d'un tableau n'est pas faite à la déclaration, il faudra alors procéder avec une boucle, case par case. Chaque valeur d'un tableau peut être initialisée individuellement.

Les éléments du tableau sont rangés en mémoire de façon consécutive.

Exemple : Considérons le programme suivant :

```
void main()
{
    UI i;
    int iTb[10] = {1,2,3,4,5,6,7,8,9,10}; // Combien d'octets?

    iTb[7] = iTb[4] * 3; // Met (5 * 3) à la place de 8.

    printf("%X,%X,%X,%X,%X,%X.\n", iTb[9], iTb[8], iTb[2], iTb[7], iTb[1], iTb[0]);

    for(i=0; i<10; i++) //
    { //
        iTb[i] = 0; // Initialise le tableau à 0.
    } //
}
```

Résultat : A, 9, 3, F, 2, 1.

2.3 Tableau à plusieurs dimensions.

Un tableau peut avoir plusieurs dimensions, exemple : 3 x 2 = 3 lignes par 2 colonnes.

```
UC ucTab[3][2] = {{1,2}, {3,4}, {5,6}}; // Initialise le Tableau.
printf("\n%u ", (int) ucTab[1][1]); // Affiche : 4
printf("\n%u ", (int) ucTab[2][1]); // Affiche : 6
printf("\n%u ", (int) ucTab[1][2]); // Affiche : 5 // Incorrecte
printf("\n%u ", (int) ucTab[3][1]); // Affiche : ? // hors zone.
```

Et même plus, 2 x 3 x 4 (et même 500 x 7000 x 23 x 1000 etc...)

```
UC ucTab[2][3][4] = { { { 1, 2, 3, 4}, { 5, 6, 7, 8}, { 9,10,11,12} },
                      { {13,14,15,16}, {17,18,19,20}, {21,22,23,24} } };
```

Les accolades permettent au programmeur de mieux visualiser les données, ils ne sont pas obligatoires, mais ils peuvent être mal placés. On ne peut pas faire ce qui suit :

```
unsigned char ucTab[3][2] = {{1,2},{3},{4,5,6}};
unsigned char ucTab[2][3] = {{1,2},{3,4},{5,6}};
```

Vous obtiendrez le message: error C242: 'array[]': too many initializers

Par contre, ceci est correct:

```
unsigned char ucTab[3][2] = {1,2,3,4,5,6};
```

3. Les chaines de caractères.

Les chaines de caractères sont un type de tableau qui contient des variables de type caractères pour former un mot ou une phrase. Pour afficher une chaine de caractères avec "printf" on utilise **%s** :

Exemple 1:

```
UC ucChaine1[] = "Salut";
UC ucChaine2[] = "Bonjour";

printf("\n%s",ucChaine1); // Affiche : Salut.
printf("\n%s",ucChaine2); // Affiche : Bonjour.
```

Note: Un tableau formé d'une chaine de caractères, contient le nombre de caractères de la **chaîne + 1** car le compilateur **ajoute toujours un zéro ('\0')** à la fin pour être capable de retrouver la fin de la chaine lors de la manipulation de cette dernière. Ainsi dans l'exemple précédant ucChaine1 contient 6 octets et ucChaine2 en contient 8. Si vous voulez travailler sur les chaines de caractères avec les fonctions du compilateurs, vous devez inclure la librairie <string.h>.

Il est possible de mettre des messages dans des tableaux à plusieurs dimensions pour pouvoir sélectionner le message avec un indice, mais pour cela vous devez avoir des messages avec le même nombre de caractère. Dans ce cas vous devez absolument indiquer les dimensions du tableau et n'oubliez pas d'ajouter un espace pour le '\0', sinon le printf ne saura pas où se termine la chaine et il affichera n'importe quoi.

Vous devez soit, placer des espaces à la fin des chaines pour les compléter.

Exemple 2:

```
UC ucMessage[3][10] = {"Salut  ", //
                        "Bonjour ", //
                        "Bye Bye !"}; // 9 caracteres + 1 pour le zero = 10.

for(i=0; i<3; i++)
{
    printf("\n%s",ucMessage[i]); // Affiche : Salut____
                                //          Bonjour__
                                //          Bye Bye !
}
```

Ou, vous devez inclure vous-même le caractère de fin de chaine ('\0').

Exemple 3:

```
UC ucMessage[3][10] = {"Salut\0", //
                        "Bonjour\0", //
                        "Bye Bye !"}; // 9 caracteres + 1 pour le zero = 10.

for(i=0; i<3; i++)
{
    printf("\n%s",ucMessage[i]); // Affiche : Salut
                                //          Bonjour
                                //          Bye Bye !
}
```

4. Les Claviers

4.1 Lecture du clavier de l'ordinateur (PC):

À date, avec notre kit STM32, nous avons lu les touches du clavier de l'ordinateur à l'aide de la fonction `getchar( )`. Cette fonction fait l'écho du caractère entré sur l'écran de l'ordinateur.

Cette fonction attend qu'une touche soit pesée avant de nous retourner son code ASCII.

Donc nos programmes "bloquent" si on appelle `getchar( )`. Ceci n'est pas idéal pour faire du contrôle.

Il est cependant possible d'utiliser une fonction des librairies STM32 qui fera l'équivalent de `kbhit( )`. Pour notre kit les caractères tapés au clavier nous arrivent par le port série. Il s'agit donc de vérifier si le port série a reçu un caractère

Les deux principales fonctions de l'API de communication série sont les suivantes :

- `HAL_UART_Receive();`
- `HAL_UART_Transmit();`

Dans les deux cas, les paramètres d'entrée des fonctions sont :

- L'adresse du handle (`huart1`),
- un pointeur de type `uint8_t`,
- le nombre de caractères à émettre ou à recevoir,
- un timeout. Ce dernier pourra être fixé arbitrairement à 10.

Voici comment l'utiliser:

```
ucTouche = 0x00;
HAL_UART_Receive(&huart1, &ucTouche, 1, 10);
if(ucTouche != 0x00)
{
    Instructions à faire lorsqu'une touche est pesée.
}
```

Ainsi, on peut aller lire le clavier seulement si une touche a été pesée ce qui permet à notre programme de rouler sans bloquer

Pour afficher le caractère à l'écran:

```
HAL_UART_Transmit(&huart1, &ucTouche, 1, 10);
```

4.2 Clavier à brancher.

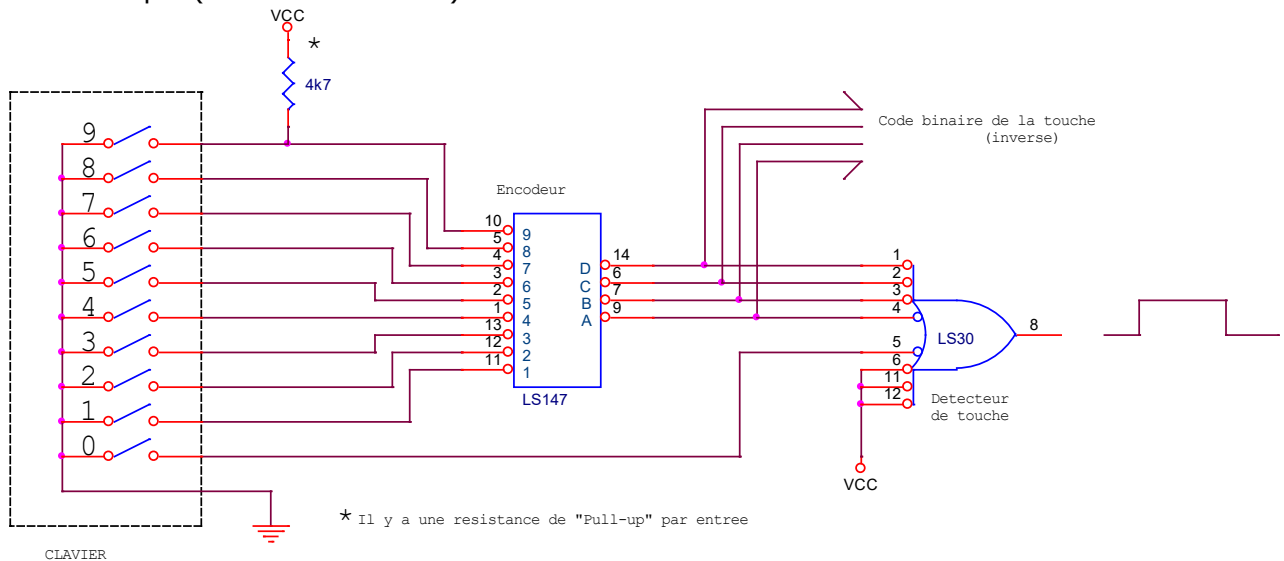
Souvent dans une application de contrôle, aucun ordinateur n'est branché sur notre kit. Si on veut entrer des informations il nous faudra un autre moyen que le clavier du PC.

Il y a, évidemment, les boutons mais on est vite limité. Le clavier à matrice comportant 10, 12, 16 touches et même plus, sera un outil très intéressant.

Principe:

Un clavier multitouche comporte plusieurs broches. Il est possible avec des circuits électroniques simples de réduire ce nombre. Ceci nous permettra de les brancher sur un processeur sans trop utiliser de bits de port.

Exemple (clavier 10 touches):



L'encodeur sort la valeur 0xF si aucune touche n'est pesée. Si on pèse sur une des touches (1 à 9), on retrouve à la sortie du LS147 la valeur binaire inversée de la touche

Aussitôt qu'une des touches (0 à 9) est pesée, le détecteur de touche sort un 1

Il nous reste à brancher les 5 fils sur un port du processeur et à vérifier si la sortie du LS30 passe à 1.

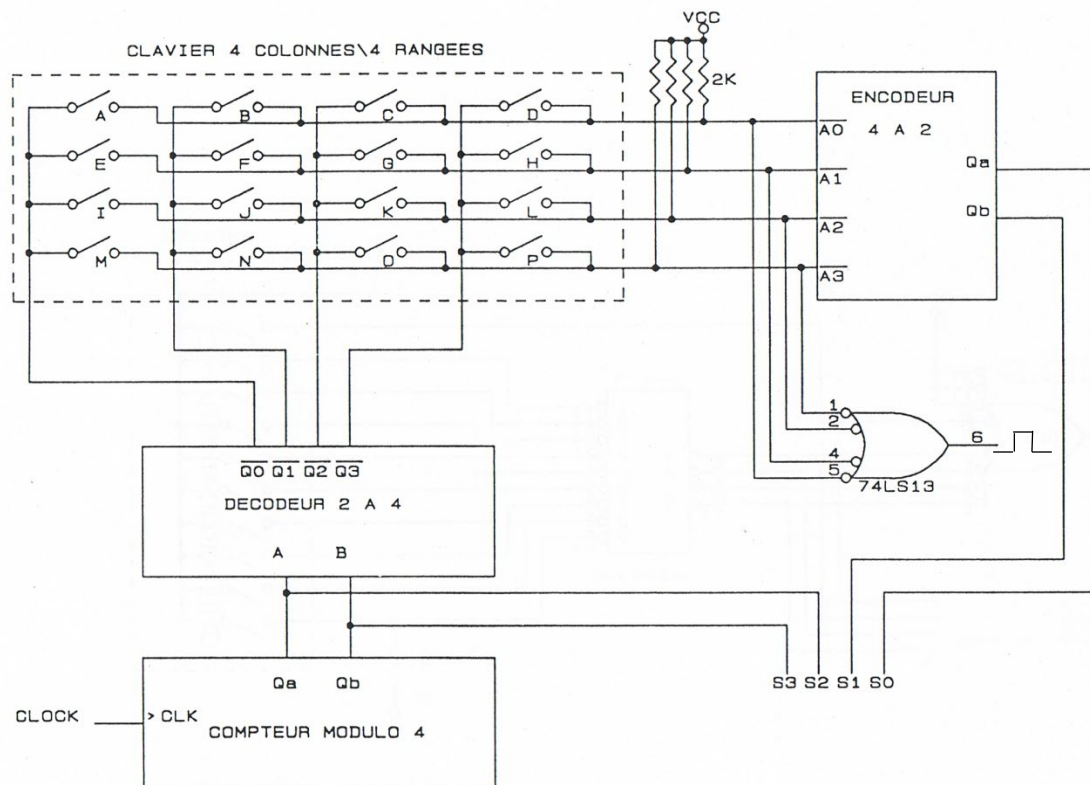
Ceci indique qu'une touche est pesée, il nous reste à lire la sortie du LS147 et à l'inverser pour avoir le numéro de la touche

4.3 Clavier à matrice:

4.3.1 Décodé avec des circuits de bases.

Un clavier à matrice comporte plusieurs touches qui sont disposées en lignes et en colonne. Dans l'exemple suivant le clavier a 16 touches organisées en 4 colonnes et 4 lignes donc 8 broches. Ici aussi, avec des circuits logiques de base, il est possible de réduire le nombre de broches à brancher au processeur.

Exemple (clavier 16 touches):



Ici, la sortie du LS13 indique qu'une touche a été pesée (pulse à 1).

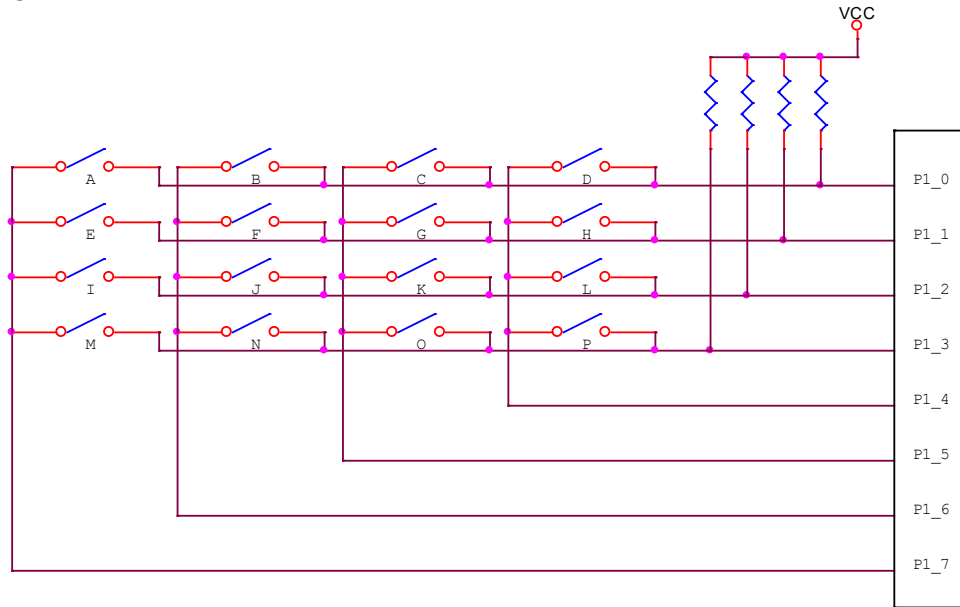
Le numéro de la touche se trouve à la sortie du compteur modulo 4 (S3 et S2 numéro de la colonne) et de l'encodeur 4 à 2 (S1 et S0 numéro de la ligne)

Exercices: Quel est le code de la touche 'B'?

Quel est le code de la touche 'L'?

4.3.2 Décodé avec un port du microcontrôleur.

Il est possible de remplacer les 4 circuits de l'exemple précédent avec un seul port du STM32



Le principe ressemble au circuit précédent.

On va mettre une colonne à 0 et vérifier si une ligne est à 0. Ceci va indiquer qu'une touche a été pesée. Si aucune touche n'est pesée, on met la colonne suivante à 0 et ainsi de suite pour les 4 colonnes. Le tout pourrait être fait dans une fonction `uclireClavier()`.

Fonctionnement de la fonction:

- Si une touche est pesée, la fonction retourne son code (0 à 15 ou encore le code ASCII).
- Si aucune touche n'est pesée, la fonction retourne un code spécial (exemple 0xFF).
- Si plus d'une touche est pesée, la fonction fait comme si aucune touche n'est pesée.

Si on veut retourner le code ASCII, on peut se servir d'un tableau comme celui-ci en utilisant le numéro de la touche pour aller chercher un élément dans le tableau.

```
UC code ucTouche[4][4]={ '1','2','3','A',           // Definition des touches
                          '4','5','6','B',           // du clavier.
                          '7','8','9','C',
                          '*', '0', '#', 'D' };
```

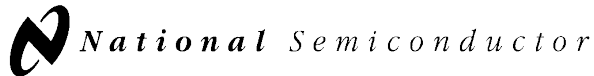
ou

```
UC code ucTouche[16] = { '1','2','3','A',           // Definition des touches
                          '4','5','6','B',           // du clavier.
                          '7','8','9','C',
                          '*', '0', '#', 'D' };
```

4.3.3 Décodé avec un 74C922.

Le 74C922 est un circuit utilisé pour décoder un clavier à matrice pouvant aller jusqu'à 16 touches.

Il contient une horloge interne, des résistances de pull-up internes, un circuit d'anti rebondissement, une sortie 3-states pouvant être relié à un bus de donnée, une sortie indiquant qu'une touche a été pesée, un registre interne qui mémorise la dernière touche (même si elle est relâchée).



MM54C922/MM74C922 16-Key Encoder

General Description

These CMOS key encoders provide all the necessary logic to fully encode an array of SPST switches. The keyboard scan can be implemented by either an external clock or external capacitor. These encoders also have on-chip pull-up devices which permit switches with up to 50k ohm resistance to be used. No diodes in the switch array are needed to eliminate ghost switches. The internal debounce circuit needs only a single external capacitor and can be defeated by omitting the capacitor. A Data Available output goes to a high level when a valid keyboard entry has been made. The Data Available output returns to a low level when the entered key is released, even if another key is depressed. The Data Available will return high to indicate acceptance of the new key after a normal debounce period; this two-key roll-over is provided between any two switches.

An internal register remembers the last key pressed even after the key is released. The TRI-STATE outputs provide for easy expansion and bus operation and are LPTTL compatible

Features

- 50 kΩ maximum switch on resistance
- On or off chip clock
- On-chip row pull-up devices
- 2 key roll-over
- Keybounce elimination with single capacitor
- Last key register at outputs
- TRI-STATE outputs LPTTL compatible
- Wide supply range 3V to 15V
- Low power consumption

Connection Diagrams

Pin Assignment for
Dual-In-Line Package

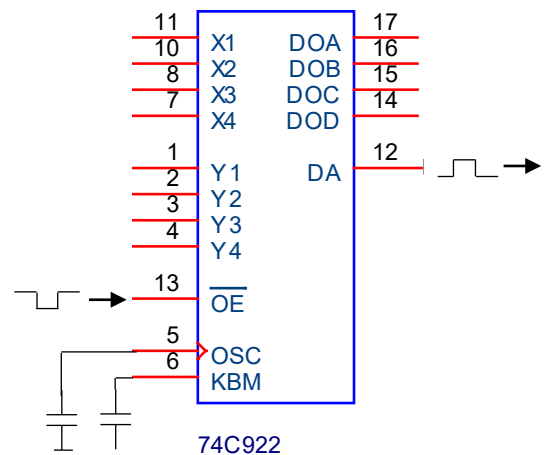
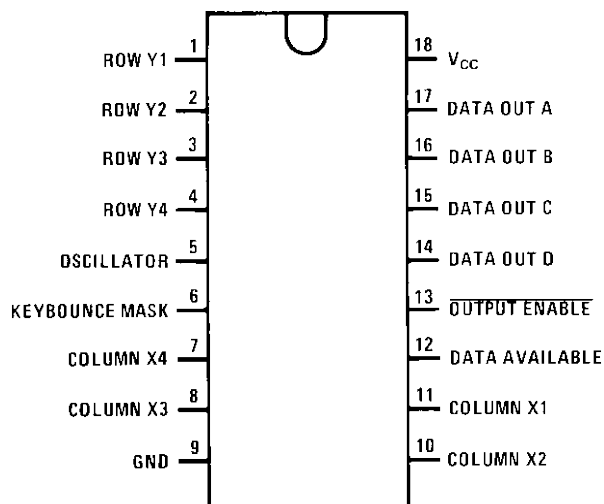


Diagramme Bloc (922).

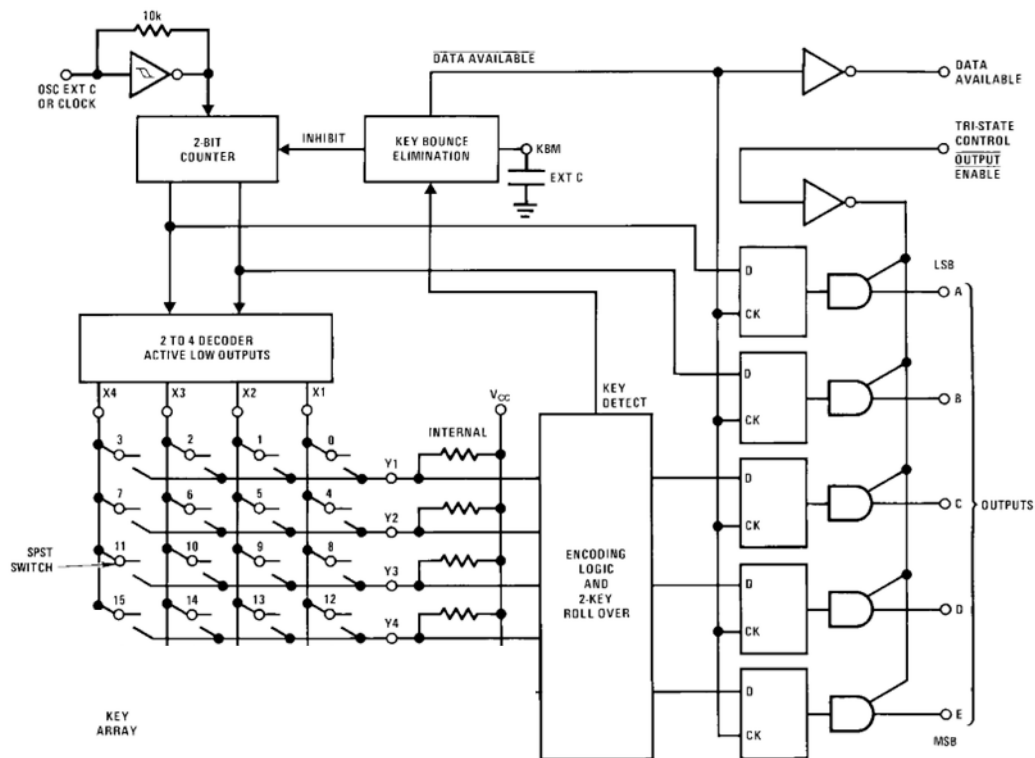
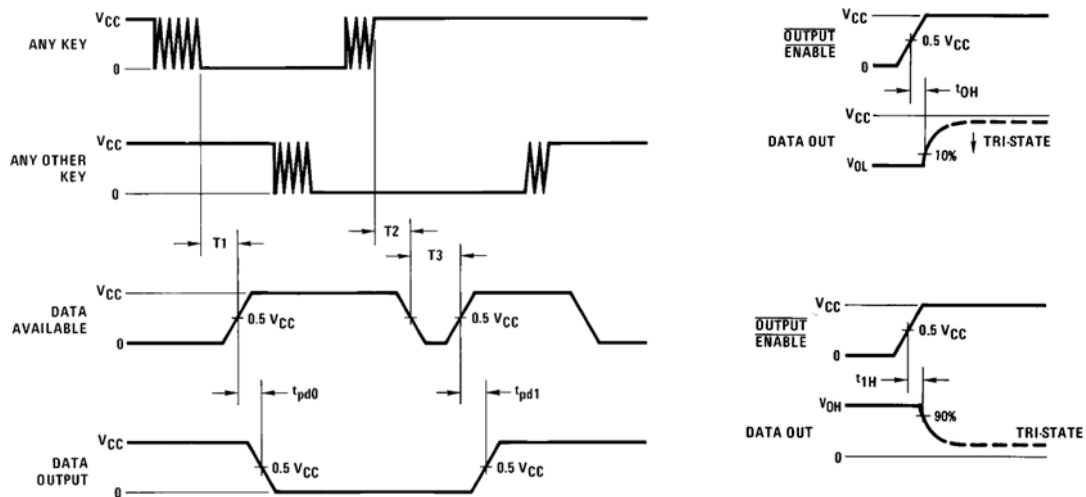


Table de vérité (922):

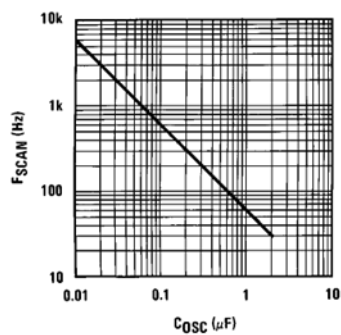
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	Y1,X1	Y1,X2	Y1,X3	Y1,X4	Y2,X1	Y2,X2	Y2,X3	Y2,X4	Y3,X1	Y3,X2	Y3,X3	Y3,X4	Y4,X1	Y4,X2	Y4,X3	Y4,X4
A	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
B	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
C	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
D	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

Formes d'ondes:

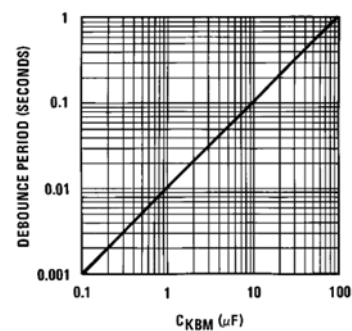


Caractéristiques typiques:

Typical F_{SCAN} vs C_{OSC}

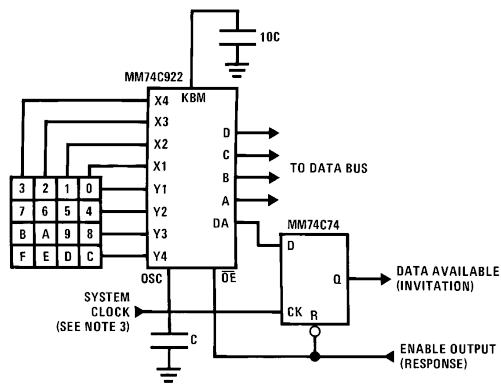


Typical Debounce Period vs C_{KBM}

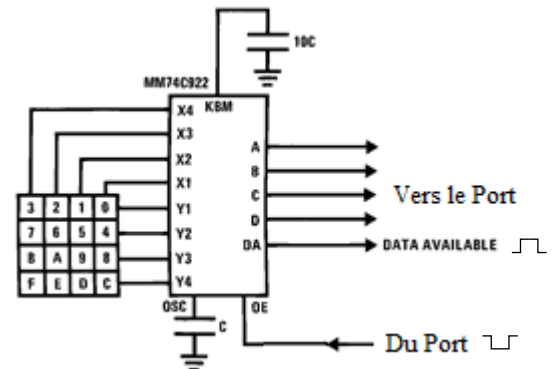


Applications typiques:

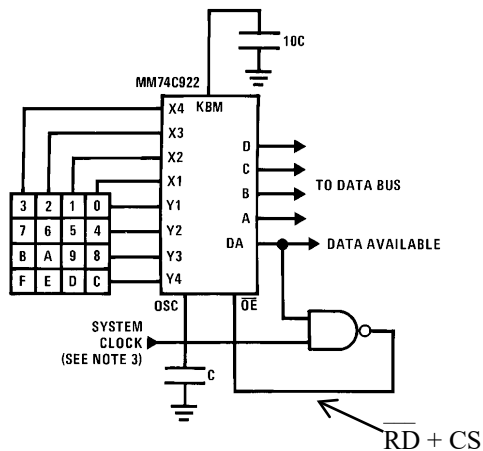
Échange synchrone



Accès avec polling (Port)



Accès synchrone au BUS



Accès asynchrone au BUS

